

Interface باعث میشه کلاس بتونه "چندین نقش" رو بازی کنه، بدون اینکه بقیه از نقش‌های دیگه‌اش خبر داشته باشن.

جالبه نه؟ سوال دیگه‌ای هست یا بریم ادامه بدیم؟ 😊

نه ممنون وایسا لینک بفرستم

چشم، منتظرم لینک رو بفرستی. 🙏

<https://refactoring.guru/duplicate-observed-data>

.Link reading is unavailable in Expert Mode. Please use Instant Mode

Duplicate Observed Data

تعریف ساده

وقتی یه کلاس هم منطق تجاری (Business Logic) داره، هم منطق رابط کاربری (UI)، داده‌های UI رو ببر به یه Domain Object جداگانه و یه کپی از داده‌ها رو در UI نگه دار.

مشکل چیه؟

این مشکل بیشتر توی برنامه‌های دسکتاپی (مثل WPF، WinForms) یا وب با state پیش میاد:

اشکال	توضیح
مسئولیت مخلوط	کلاس هم محاسبات تجاری داره، هم رنگ دکمه و اندازه فرم
تست ناپذیری	برای تست منطق تجاری، باید UI رو هم راه بندازی
وابستگی به UI Framework	منطق تجاری به کنترل‌های UI (مثل Grid، TextBox) وابسته میشه

مثال قبل از ریفکتور

csharp

```

// این کلاس هم داده داره، هم UI
public class CustomerForm : Form
{
    ;private TextBox nameTextBox
    ;private Label resultLabel

    public string CustomerName
    {
        ;get => nameTextBox.Text
        ;set => nameTextBox.Text = value
    }

    ()public decimal CalculateDiscount
    {
        // این منطق تجاریه، ولی توی Form هست!
        if (CustomerName.Length > 10)
            ;return 0.1m
            ;return 0.05m
    }

    private void ButtonCalculate_Click(object sender, EventArgs e)
    {
        ;()decimal discount = CalculateDiscount
        ;"{discount:P0} تخفیف: $" = resultLabel.Text
    }
}

```

بعد از Duplicate Observed Data

csharp

```

// قدم ۱: Domain Object (فقط منطق تجاری)
public class Customer
{
    public string Name { get; set; }

    ()public decimal CalculateDiscount

```

```

    }

    if (Name?.Length > 10)
        ;return 0.1m
        ;return 0.05m
    {
        {

            // قدم ۲: UI (فقط نمایش)
            public class CustomerForm : Form
            {
                ;private TextBox nameTextBox
                ;private Label resultLabel
                Domain Object از کپی // ;private Customer _customer

                ()public CustomerForm
                {
                    ;()customer = new Customer_
                    {

                        // همگامسازی UI ← Domain
                        ()private void SyncFromDomain
                        {
                            ;nameTextBox.Text = _customer.Name
                        {

                            // همگامسازی Domain ← UI
                            ()private void SyncToDomain
                            {
                                ;customer.Name = nameTextBox.Text_
                                {

                                    private void ButtonCalculate_Click(object sender, EventArgs e)
                                    {
                                        // اول داده از UI به Domain میره
                                        Domain در محاسبه // ;()decimal discount = _customer.CalculateDiscount
                                        ;"{discount:P0} تخفیف: $" = resultLabel.Text
                                    {
                                        {

```

مزیت

توضیح

تست مستقل	Customer.CalculateDiscount رو بدون UI تست می‌کنی
جدایی مسئولیت‌ها	UI فقط نمایشه، Domain فقط منطق
قابل استفاده مجدد	Customer رو می‌تونی توی Web API یا Console هم استفاده کنی
تغییر UI آسون	از WinForms به WPF بری، Customer دست نخورده می‌مونه

الگوی Observer (اختیاری)

وقتی Domain تغییر می‌کنه، UI باید بروز بشه (و برعکس). این کار معمولاً با **Data Binding** یا **Observer Pattern** انجام میشه:

csharp

```
// INotifyPropertyChanged با Domain Object
public class Customer : INotifyPropertyChanged
{
    private string _name
    public string Name
    {
        get => _name
        set { _name = value; NotifyPropertyChanged(); }
    }

    public event PropertyChangedEventHandler PropertyChanged
    private void NotifyPropertyChanged([CallerMemberName] string propertyName = "")
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName))
    }
}
```

جمع‌بندی درمان‌های Large Class

کاربرد	تکنیک	#
جدا کردن مسئولیت‌های مختلف	Extract Class	۱
حالت‌های خاص با ارث‌بری	Extract Subclass	۲
اشتراک رفتار بین کلاس‌های مختلف	Extract Interface	۳
جدا کردن منطق تجاری از UI	Duplicate Observed Data	۴

Large Class بسته شد! 🍌

حالا بریم سراغ بوی بد بعدی توی **Bloaters: Primitive Obsession**

بفرما لینکش رو:



<https://refactoring.guru/smells/primitive-obsession>